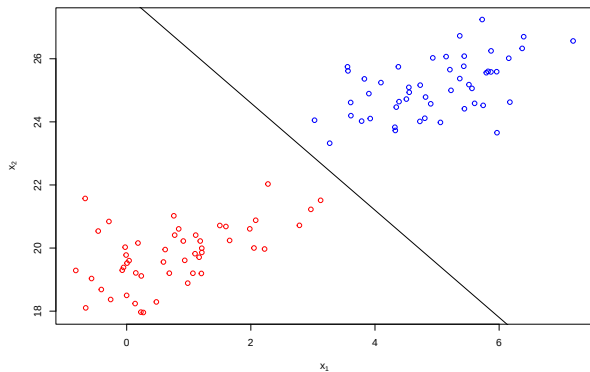


Support Vector Machines (SVM)

Support vector machine

- ▶ A classification method. We focus on the case with two classes.
- ▶ Maximal margin classifier: Separate classes completely with linear boundaries
- ▶ Support vector classifier: Complete separation is not necessary
- ▶ Support vector machines: Also allow for non-linear boundaries

Maximal margin classifier



Hyperplanes

- ▶ Example: $p = 2$ (2 predictors). A hyperplane is defined by

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$$

$$\Longleftrightarrow$$

$$X_2 = -\frac{\beta_0}{\beta_2} - \frac{\beta_1}{\beta_2} X_1 \quad \square$$

- ▶ If $p = 3$ an hyperplane is given by

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 = 0$$

and is a plane (2-dimensional object)

- ▶ In general, a hyperplane in a p -dimensional space is a $p - 1$ -dimensional object which separates the p -dimensional space into two pieces.

Classification by separating hyperplanes

- ▶ We have a data matrix with n training observations and p variables (predictors)

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$

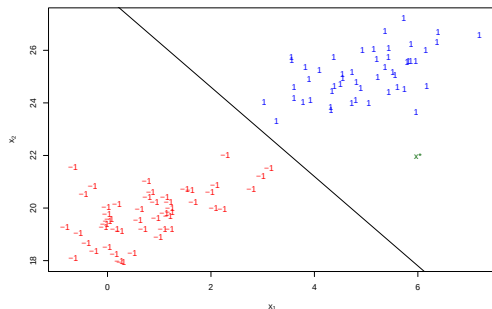
- ▶ Each observation/individual i , is represented by a row

$$\mathbf{x}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{ip} \end{bmatrix}, \text{ for } i = 1, 2, \dots, n$$

- ▶ Each observation falls into one of two classes, represented by $y_i \in \{-1, 1\}$

Classification by separating hyperplanes

We would like to classify a test observation $\mathbf{x}^*$ (p -vector)



The observation with $\mathbf{x}^*$ is predicted to belong to class 1 since

$$x_2^* > -\frac{\beta_0}{\beta_2} - \frac{\beta_1}{\beta_2}x_1^*$$

which is equivalent with

$$\beta_0 + \beta_1x_1^* + \beta_2x_2^* > 0$$

Classification by separating hyperplanes

In general predict

- ▶ $y_i = 1$ if

$$f(\mathbf{x}^*) = \beta_0 + \beta_1 x_{i1}^* + \beta_2 x_{i2}^* + \dots + \beta_p x_{ip}^* > 0$$

- ▶ and $y_i = -1$ if

$$f(\mathbf{x}^*) = \beta_0 + \beta_1 x_{i1}^* + \beta_2 x_{i2}^* + \dots + \beta_p x_{ip}^* < 0$$

- ▶ This implies that a separating hyperplane has the property that

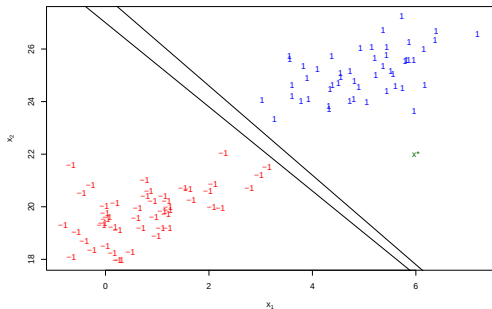
$$y_i f(\mathbf{x}^*) = y_i (\beta_0 + \beta_1 x_{i1}^* + \beta_2 x_{i2}^* + \dots + \beta_p x_{ip}^*) > 0$$

for all $i = 1, 2, \dots, n$.

- ▶ If the magnitude (absolute value) of $f(\mathbf{x}^*)$ is large we can be confident about our class assignment.

The maximal margin classifier

The separating hyperplane is not unique



- The maximal margin classifier is the separating hyperplane with the largest minimum distance M from the observations.

The maximal margin classifier

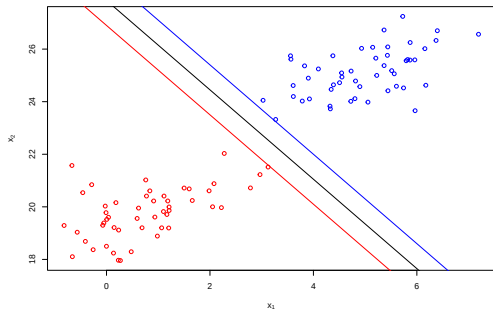


$$\left\{ \begin{array}{l} \text{maximize } M \\ \beta_0, \beta_1, \dots, \beta_p \\ \text{subject to} \\ \sum_{j=1}^p \beta_j^2 = 1 \\ \underbrace{y_i(\beta_0 + \beta_1 x_{i1}^* + \beta_2 x_{i2}^* + \dots + \beta_p x_{ip}^*)}_{\text{Perpendicular distance from hyperplane}} \geq M, \quad \forall i = 1, \dots, n \end{array} \right.$$

- ▶ The restriction $\sum_{j=1}^p \beta_j^2 = 1$ is necessary to make $y_i f(\mathbf{x}^*)$ the distance between the points and the hyperplane.

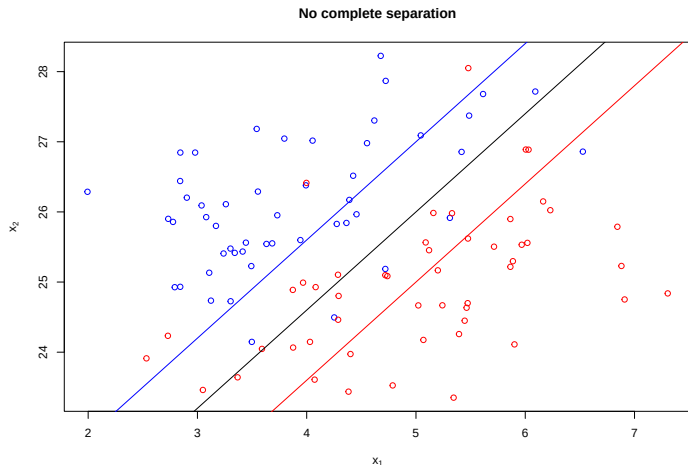
The maximal margin classifier

M is the perpendicular distance between the blue and black lines
(and red and black lines)



No separating hyperplane

This optimization problem is often not solvable, there exist no separating hyperplane. An example:



Support vector classifier (soft margin classifier)

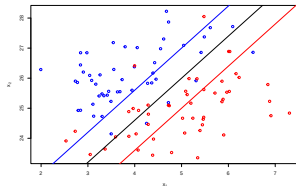
- ▶ We like to allow for some misclassified observations; allow them to be on the wrong side of the margin (red and blue line) or even on the wrong side of the separating hyperplane.



$$\left\{ \begin{array}{l} \text{maximize}_{\beta_0, \beta_1, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n} M \\ \text{subject to} \\ \sum_{j=1}^p \beta_j^2 = 1 \\ y_i(\beta_0 + \beta_1 x_{i1}^* + \beta_2 x_{i2}^* + \dots + \beta_p x_{ip}^*) \geq M(1 - \epsilon_i), \quad i = 1, \dots, n \\ \epsilon_i \geq 0, \quad \sum_{i=1}^n \epsilon_i \leq C \end{array} \right.$$

- ▶ C is a tuning parameter that can be found by...
- ▶ Classification is done as it was for the maximal margin classifier.

Support vector classifier



- ▶ Only observations violating the margin, or on the margin, affect the solution
- ▶ The *slack variables* $\epsilon_1, \dots, \epsilon_n$ allows the observations to be on the wrong side of the margins or hyperplane.
- ▶ They tell us where the observations are located.
- ▶ $\epsilon_i = 0$ means that the i th observation is on the correct side of the margin.
- ▶ $\epsilon_i > 0$ means that the i th observation is on the wrong side of the margin.

The constant C regulates how much the margin can be violated

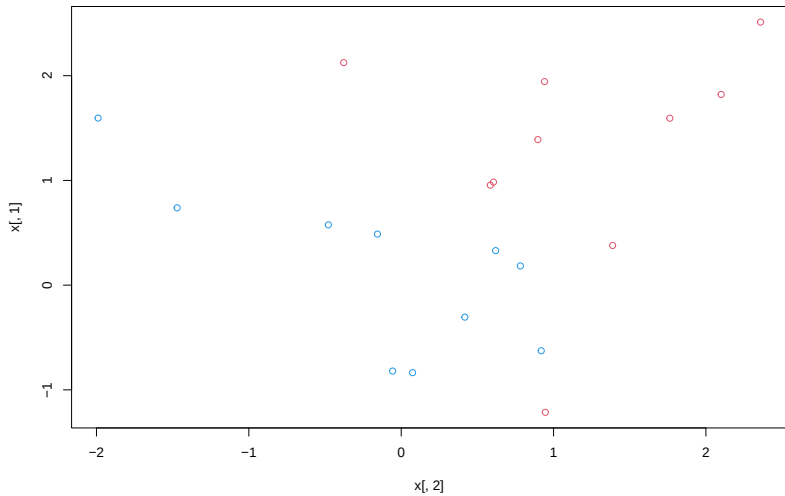
- ▶ If $C = 0$ we do not allow for any violations and we have the maximal margin classifier.
- ▶ If $C > 0$, at most C observations can be on the wrong side of the hyperplane.
- ▶ A large C means that we allow more violations; we have a wide margin.
- ▶ C is a tuning parameter that can be found by cross-validation.

A simulated dataset (page 359 in ISLR)

```
set.seed(1)
x=matrix(rnorm(20*2),ncol=2)
y=c(rep(-1,10),rep(1,10))
x[y==1,]=x[y==1,]+1
head(cbind(x,y),16)
```

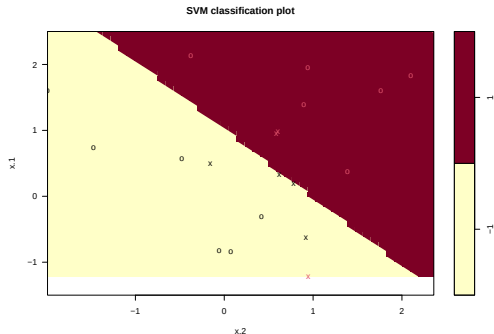
```
##                               y
## [1,] -0.6264538  0.91897737 -1
## [2,]  0.1836433  0.78213630 -1
## [3,] -0.8356286  0.07456498 -1
## [4,]  1.5952808 -1.98935170 -1
## [5,]  0.3295078  0.61982575 -1
## [6,] -0.8204684 -0.05612874 -1
## [7,]  0.4874291 -0.15579551 -1
## [8,]  0.7383247 -1.47075238 -1
## [9,]  0.5757814 -0.47815006 -1
## [10,] -0.3053884  0.41794156 -1
## [11,]  2.5117812  2.35867955  1
## [12,]  1.3898432  0.89721227  1
## [13,]  0.3787594  1.38767161  1
## [14,] -1.2146999  0.94619496  1
## [15,]  2.1249309 -0.37705956  1
## [16,]  0.9550664  0.58500544  1
```

A simulated dataset



A simulated dataset

```
library(e1071)
dat=data.frame(x=x,y=as.factor(y))
svmfit=svm(y~.,data=dat,kernel="linear",
           ,cost=10,scale=FALSE)
plot(svmfit,dat,ylim=c(-1.5,2.5))
```



Support vector machines

Consider the support vector classifier problem again

$$\left\{ \begin{array}{l} \text{maximize } M \\ \beta_0, \beta_1, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n \\ \text{subject to} \\ \sum_{j=1}^p \beta_j^2 = 1 \\ y_i (\underbrace{\beta_0 + \beta_1 x_{i1}^* + \beta_2 x_{i2}^* + \dots + \beta_p x_{ip}^*}_{f(\mathbf{x}^*)}) \geq M(1 - \epsilon_i), \quad i = 1, \dots, n \\ \epsilon_i \geq 0, \quad \sum_{i=1}^n \epsilon_i \leq C \end{array} \right.$$

The decision boundary, $f(\mathbf{x}^*)$, must not necessarily be linear in $\mathbf{x}^*$.

Support vector machines

We let

$$f(\mathbf{x}^*) = \beta_0 + \sum_{i=1}^n \alpha_i K(\mathbf{x}^*, \mathbf{x}_i),$$

where K is a *kernel*.

Two commonly used kernels are the *polynomial kernel* of degree d

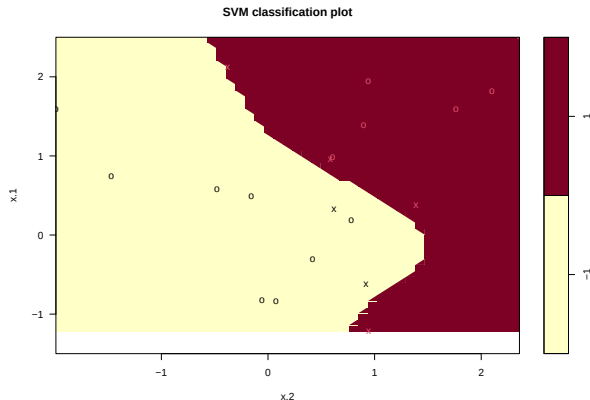
$$K(\mathbf{x}_i, \mathbf{x}_{i'}) = \left(1 + \sum_{j=1}^p x_{ij} x_{i'j} \right)^d$$

and the *radial kernel*

$$K(\mathbf{x}_i, \mathbf{x}_{i'}) = \exp \left(-\gamma \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right)$$

A simulated dataset

```
library(e1071)
dat=data.frame(x=x,y=as.factor(y))
svmfit=svm(y~.,data=dat,kernel="polynomial",
           ,cost=10,scale=FALSE)
plot(svmfit,dat,ylim=c(-1.5,2.5))
```



A simulated dataset

```
library(e1071)
dat=data.frame(x=x,y=as.factor(y))
svmfit=svm(y~.,data=dat,kernel="radial",
           ,cost=10,scale=FALSE)
plot(svmfit,dat,ylim=c(-1.5,2.5))
```

